

```
import numpy as np
arreglo= np . array ([10 , 20, 30 , 40 ,50])
suma = 0

for valor in arreglo :
    suma += valor

print (f"La suma del arreglo es: {suma}")
```

La suma del arreglo es: 150

```
matriz = [
[1, 2 , 3],
[4, 5 , 6],
[7, 8 , 9],

]

for fila in matriz:
    for elemento in fila:
        print(elemento , end = " ")
    print () # Salto de línea al terminar cada fila
```

1 2 3
4 5 6
7 8 9

```
for fila in matriz:
    print(fila)
```

[1, 2, 3]
[4, 5, 6]
[7, 8, 9]

```
matriz = [
    [1, 2 , 3],
    [4, 5 , 6],
    [7, 8 , 9]
]

for fila in matriz:
    for elemento in fila:
        print(elemento, end = " ")
```

1 2 3 4 5 6 7 8 9

```
MS = [
    [3, 5 , 4],
    [-1, 2 , 7],
]
```

```
print(MS)
```

[[3, 5, 4], [-1, 2, 7]]

```
for fila in MS:
    for elemento in fila:
        print(elemento, end = " ")
    print()
```

3 5 4
-1 2 7

```
print(MS[0][1])
```

5

```
def luisa_det2x2(A):
    a=A[0][0]
    b=A[0][1]
    c=A[1][0]
    d=A[1][1]
```

```
y=a*d-b*c  
return y
```

```
A3=[  
    [1, 3],  
    [4, 5],  
]  
print(A3)  
luisa_det2x2(A3)
```

```
[[1, 3], [4, 5]]  
-7
```

```
def luisa_det3x3(B):  
    a=B[0][0]  
    b=B[0][1]  
    c=B[0][2]  
    d=B[1][0]  
    e=B[1][1]  
    f=B[1][2]  
    g=B[2][0]  
    h=B[2][1]  
    i=B[2][2]  
    B1=[  
        [e,f],  
        [h,i],  
    ]  
  
    B2=[  
        [d,f],  
        [g,i],  
    ]  
  
    B3=[  
        [d,e],  
        [g,h],  
    ]  
  
    y=a*luisa_det2x2(B1)-b*luisa_det2x2(B2)+c*luisa_det2x2(B3)  
    return y
```

```
Z = [  
    [1, 2, 7],  
    [2, 3, 4],  
    [5, 6, 8]  
]  
luisa_det3x3(Z)
```

```
-13
```

```
x=luisa_det2x2([[4,2],[8,5]])/luisa_det2x2([[3,2],[-1,5]])  
print(f"El resultado de la x es {x:.3f}")
```

```
El resultado de la x es 0.235
```

Operaciones entre Matrices con NumPy en Python

Documento elaborado para el curso de Álgebra Lineal

8 de septiembre de 2026

Resumen

Este documento presenta una guía completa sobre las operaciones entre matrices utilizando la librería NumPy en Python. Se describen las principales operaciones algebraicas con matrices, incluyendo suma, resta, multiplicación, producto punto, transpuesta, inversa, determinante, y operaciones elemento a elemento. Cada operación viene acompañada de ejemplos prácticos con código y salidas esperadas.

Índice

1. Introducción a NumPy	2
1.1. Creación de Matrices	2
2. Operaciones Básicas entre Matrices	2
2.1. Suma de Matrices	2
2.2. Resta de Matrices	2
2.3. Multiplicación por un Escalar	3
3. Producto de Matrices	3
3.1. Producto Punto (Multiplicación Matricial)	3
3.2. Multiplicación Elemento a Elemento	4
4. Operaciones con Matrices Cuadradas	4
4.1. Transpuesta de una Matriz	4
4.2. Matriz Inversa	4
4.3. Determinante de una Matriz	5
5. Otras Operaciones Importantes	5
5.1. Traza de una Matriz	5
5.2. Norma de una Matriz	6
5.3. Autovalores y Autovectores	6
5.4. Resolución de Sistemas de Ecuaciones Lineales	6
6. Ejemplos Adicionales	7
6.1. Operaciones con Matrices Rectangulares	7
6.2. Operaciones con Matrices Complejas	7
7. Conclusión	8

1. Introducción a NumPy

NumPy (Numerical Python) es una librería fundamental para el cálculo científico en Python. Proporciona soporte para arreglos multidimensionales y matrices, junto con una colección de funciones matemáticas para operar con estos arreglos.

Para trabajar con matrices en NumPy, primero debemos importar la librería:

```
1 import numpy as np
```

1.1. Creación de Matrices

Existen varias formas de crear matrices en NumPy:

```
1 # Crear una matriz a partir de una lista
2 A = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
3
4 # Crear una matriz de ceros
5 B = np.zeros((3, 3))
6
7 # Crear una matriz de unos
8 C = np.ones((2, 3))
9
10 # Crear una matriz identidad
11 I = np.eye(3)
12
13 # Crear una matriz con valores aleatorios
14 D = np.random.rand(3, 2)
```

2. Operaciones Básicas entre Matrices

2.1. Suma de Matrices

La suma de matrices se realiza elemento a elemento. Las matrices deben tener la misma dimensión.

```
1 # Ejemplo de suma de matrices
2 A = np.array([[1, 2], [3, 4]])
3 B = np.array([[5, 6], [7, 8]])
4
5 C = A + B
6 print("A + B =")
7 print(C)
8
9 # Salida:
10 # A + B =
11 # [[ 6  8]
12 #  [10 12]]
```

2.2. Resta de Matrices

La resta de matrices también se realiza elemento a elemento.

```
1 # Ejemplo de resta de matrices
2 A = np.array([[10, 20], [30, 40]])
3 B = np.array([[1, 2], [3, 4]])
```

```

4
5 C = A - B
6 print("A - B =")
7 print(C)
8
9 # Salida:
10 # A - B =
11 # [[ 9 18]
12 #  [27 36]]

```

2.3. Multiplicación por un Escalar

Multiplicar una matriz por un escalar implica multiplicar cada elemento por ese escalar.

```

1 # Ejemplo de multiplicación por escalar
2 A = np.array([[1, 2], [3, 4]])
3 k = 3
4
5 B = k * A
6 print("3 * A =")
7 print(B)
8
9 # Salida:
10 # 3 * A =
11 # [[ 3  6]
12 #  [ 9 12]]

```

3. Producto de Matrices

3.1. Producto Punto (Multiplicación Matricial)

El producto punto o multiplicación matricial se realiza con el operador `@` o la función `np.dot()`. Para que dos matrices se puedan multiplicar, el número de columnas de la primera debe ser igual al número de filas de la segunda.

```

1 # Ejemplo de producto punto
2 A = np.array([[1, 2, 3], [4, 5, 6]])
3 B = np.array([[7, 8], [9, 10], [11, 12]])
4
5 # Usando el operador @
6 C = A @ B
7 print("A @ B =")
8 print(C)
9
10 # Usando np.dot()
11 D = np.dot(A, B)
12 print("np.dot(A, B) =")
13 print(D)
14
15 # Salida:
16 # A @ B =
17 # [[ 58  64]
18 #  [139 154]]
19 # np.dot(A, B) =

```

```

20 # [[ 58  64]
21 #  [139 154]]

```

3.2. Multiplicación Elemento a Elemento

También conocida como producto de Hadamard, se realiza con el operador `*`.

```

1 # Ejemplo de multiplicación elemento a elemento
2 A = np.array([[1, 2], [3, 4]])
3 B = np.array([[5, 6], [7, 8]])
4
5 C = A * B
6 print("A * B (elemento a elemento) =")
7 print(C)
8
9 # Salida:
10 # A * B (elemento a elemento) =
11 # [[ 5 12]
12 #  [21 32]]

```

4. Operaciones con Matrices Cuadradas

4.1. Transpuesta de una Matriz

La transpuesta de una matriz se obtiene intercambiando filas por columnas.

```

1 # Ejemplo de transpuesta
2 A = np.array([[1, 2, 3], [4, 5, 6]])
3
4 A_T = A.T
5 print("A =")
6 print(A)
7 print("A^T =")
8 print(A_T)
9
10 # Salida:
11 # A =
12 # [[1 2 3]
13 #  [4 5 6]]
14 # A^T =
15 # [[1 4]
16 #  [2 5]
17 #  [3 6]]

```

4.2. Matriz Inversa

La inversa de una matriz existe solo si es cuadrada y su determinante es diferente de cero.

```

1 # Ejemplo de matriz inversa
2 A = np.array([[4, 7], [2, 6]])
3
4 A_inv = np.linalg.inv(A)
5 print("A =")
6 print(A)

```

```

7 print("A^{-1} =")
8 print(A_inv)
9
10 # Verificando: A * A^{-1} = I
11 I = A @ A_inv
12 print("A * A^{-1} =")
13 print(I)
14
15 # Salida aproximada:
16 # A =
17 # [[4 7]
18 #  [2 6]]
19 # A^{-1} =
20 # [[ 0.6 -0.7]
21 #  [-0.2  0.4]]
22 # A * A^{-1} =
23 # [[1. 0.]
24 #  [0. 1.]]

```

4.3. Determinante de una Matriz

El determinante es un valor escalar que se puede calcular para matrices cuadradas.

```

1 # Ejemplo de determinante
2 A = np.array([[3, 1], [2, 4]])
3
4 det_A = np.linalg.det(A)
5 print("A =")
6 print(A)
7 print("det(A) =", det_A)
8
9 # Salida:
10 # A =
11 # [[3 1]
12 #  [2 4]]
13 # det(A) = 10.0

```

5. Otras Operaciones Importantes

5.1. Traza de una Matriz

La traza es la suma de los elementos de la diagonal principal.

```

1 # Ejemplo de traza
2 A = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
3
4 traza = np.trace(A)
5 print("A =")
6 print(A)
7 print("tr(A) =", traza)
8
9 # Salida:
10 # A =
11 # [[1 2 3]
12 #  [4 5 6]
13 #  [7 8 9]]

```

```
14 # tr(A) = 15
```

5.2. Norma de una Matriz

NumPy permite calcular diferentes tipos de normas.

```
1 # Ejemplo de norma de Frobenius
2 A = np.array([[1, 2], [3, 4]])
3
4 norma_frobenius = np.linalg.norm(A, 'fro')
5 print("A =")
6 print(A)
7 print("||A||_F =", norma_frobenius)
8
9 # Norma L2 (espectral)
10 norma_l2 = np.linalg.norm(A, 2)
11 print("||A||_2 =", norma_l2)
12
13 # Salida:
14 # A =
15 # [[1 2]
16 #  [3 4]]
17 # ||A||_F = 5.477225575051661
18 # ||A||_2 = 5.464985704219043
```

5.3. Autovalores y Autovectores

El cálculo de autovalores y autovectores es fundamental en álgebra lineal.

```
1 # Ejemplo de autovalores y autovectores
2 A = np.array([[4, -2], [1, 1]])
3
4 autovalores, autovectores = np.linalg.eig(A)
5 print("A =")
6 print(A)
7 print("Autovalores =", autovalores)
8 print("Autovectores =")
9 print(autovectores)
10
11 # Salida:
12 # A =
13 # [[ 4 -2]
14 #  [ 1  1]]
15 # Autovalores = [3.  2.]
16 # Autovectores =
17 # [[0.89442719  0.70710678]
18 #  [0.4472136   0.70710678]]
```

5.4. Resolución de Sistemas de Ecuaciones Lineales

NumPy proporciona funciones para resolver sistemas de ecuaciones lineales.

```
1 # Ejemplo: Resolver el sistema Ax = b
2 # 3x + y = 9
3 # x + 2y = 8
4
```



```

5 A = np.array([[3, 1], [1, 2]])
6 b = np.array([9, 8])
7
8 # Usando np.linalg.solve()
9 x = np.linalg.solve(A, b)
10 print("Soluci n del sistema Ax = b:")
11 print("x =", x)
12 print("y =", x[1])
13
14 # Verificaci n
15 print("Verificaci n: A * x =", A @ x)
16
17 # Salida:
18 # Soluci n del sistema Ax = b:
19 # x = [2. 3.]
20 # y = 3.0
21 # Verificaci n: A * x = [9. 8.]

```

6. Ejemplos Adicionales

6.1. Operaciones con Matrices Rectangulares

```

1 # Operaciones con matrices rectangulares
2 A = np.array([[1, 2, 3], [4, 5, 6]])
3 B = np.array([[7, 8], [9, 10], [11, 12]])
4
5 # Producto A * B (2x3 * 3x2 = 2x2)
6 C = A @ B
7 print("A (2x3) * B (3x2) = C (2x2)")
8 print(C)
9
10 # Transpuesta de A (2x3 -> 3x2)
11 A_T = A.T
12 print("A^T (3x2) =")
13 print(A_T)
14
15 # Salida:
16 # A (2x3) * B (3x2) = C (2x2)
17 # [[ 58  64]
18 #  [139 154]]
19 # A^T (3x2) =
20 # [[1 4]
21 #  [2 5]
22 #  [3 6]]

```

6.2. Operaciones con Matrices Complejas

```

1 # Matrices con n meros complejos
2 A = np.array([[1+2j, 3-1j], [2+0j, 4+3j]])
3
4 # Conjugada transpuesta (Hermitiana)
5 A_H = A.conj().T
6 print("A_H =")
7 print(A_H)

```

```

8 print("A^H (conjugada transpuesta) =")
9 print(A_H)
10
11 # Salida:
12 # A =
13 # [[1.+2.j 3.-1.j]
14 #  [2.+0.j 4.+3.j]]
15 # A^H (conjugada transpuesta) =
16 # [[1.-2.j 2.-0.j]
17 #  [3.+1.j 4.-3.j]]

```

7. Conclusión

NumPy proporciona una amplia gama de funciones para realizar operaciones con matrices de manera eficiente. Las principales operaciones abordadas en este documento son:

- **Operaciones aritméticas:** suma, resta y multiplicación por escalar
- **Producto matricial:** producto punto y multiplicación elemento a elemento
- **Operaciones básicas:** transpuesta, inversa y determinante
- **Operaciones avanzadas:** traza, norma, autovalores y autovectores
- **Aplicaciones prácticas:** resolución de sistemas de ecuaciones lineales

La sintaxis de NumPy es intuitiva y eficiente, lo que la convierte en una herramienta indispensable para el cálculo científico y el álgebra lineal computacional.

Referencias

1. Harris, C.R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.
2. NumPy Documentation. <https://numpy.org/doc/stable/>
3. Oliphant, T.E. (2006). *Guide to NumPy*. Trelgol Publishing.

Álgebra Lineal con Python

Fundamentos Matemáticos, Métodos Computacionales
e Implementación con NumPy y SciPy

Documentación Técnica y Manual de Referencia

8 de septiembre de 2026

Resumen

El álgebra lineal constituye la columna vertebral matemática de la computación científica contemporánea, el aprendizaje automático (*Machine Learning*), el procesamiento de imágenes, la física computacional y la optimización. Este documento expone de forma rigurosa y pedagógica las operaciones fundamentales del álgebra lineal (vectores, matrices, sistemas de ecuaciones lineales, determinantes, autovalores y descomposiciones espectrales), acoplando cada definición matemática abstracta con su correspondiente implementación práctica y vectorizada en el lenguaje de programación **Python**, empleando las bibliotecas especializadas **NumPy** y **SciPy**.

Índice

1. Introducción y Configuración del Entorno	2
1.1. Importación de Módulos y Convenciones	2
2. Vectores y Operaciones Vectoriales en $\mathbb{R}^n$	2
2.1. Creación y Propiedades Básicas	2
2.2. Suma Vectorial y Multiplicación por Escalar	3
2.3. Producto Escalar (Producto Punto)	3
2.4. Norma Euclidiana y Magnitud	3
2.5. Ángulo entre Vectores y Proyección Ortogonal	3
2.6. Producto Cruz (Vectorial en $\mathbb{R}^3$)	4
3. Matrices y Álgebra Matricial	4
3.1. Creación y Matrices Especiales	4
3.2. Multiplicación Matricial vs. Producto de Hadamard	5
3.3. Matriz Transpuesta y Traza	5
4. Determinantes, Rango e Invertibilidad	5
4.1. Determinante de una Matriz	5
4.2. Rango de una Matriz e Independencia Lineal	6
4.3. Inversa de una Matriz	6
5. Sistemas de Ecuaciones Lineales: $Ax = b$	6
5.1. Resolución Directa y Eficiencia Algorítmica	6
5.2. Condicionamiento Numérico	7
5.3. Sistemas Sobredeterminados y Mínimos Cuadrados	7
6. Valores y Vectores Propios (Autovalores y Autovectores)	7
6.1. Cálculo en Python con <code>np.linalg.eig</code>	7
6.2. Diagonalización de Matrices	8

7. Descomposiciones Matriciales Fundamentales	8
7.1. Descomposición LU	8
7.2. Descomposición QR	8
7.3. Descomposición en Valores Singulares (SVD)	9
8. Tabla Resumen de Sintaxis y Operaciones	9
9. Buenas Prácticas Numéricas y Conclusiones	10

1. Introducción y Configuración del Entorno

En el ámbito computacional, el álgebra lineal no se implementa mediante bucles explícitos (`for`), sino mediante algoritmos altamente optimizados escritos en Fortran y C (*BLAS: Basic Linear Algebra Subprograms* y *LAPACK: Linear Algebra Package*). Las bibliotecas NumPy y SciPy proporcionan interfaces de Python de alto nivel que ejecutan estas subrutinas vectorizadas directamente a velocidad de lenguaje de bajo nivel.

1.1. Importación de Módulos y Convenciones

Para todo el desarrollo matemático se utilizan las siguientes convenciones estándar:

```

1 import numpy as np
2 import scipy.linalg as la
3
4 # Configuración de impresión para claridad numérica
5 np.set_printoptions(precision=4, suppress=True)
```

- `np.ndarray`: Es la estructura troncal que modela tanto vectores (1D) como matrices (2D) y tensores (n D).
- `scipy.linalg`: Submódulo que contiene funciones avanzadas que extienden las capacidades de `numpy.linalg`.

2. Vectores y Operaciones Vectoriales en $\mathbb{R}^n$

Definición Matemática: Vector

Un vector $\mathbf{v}$ en el espacio euclidiano $\mathbb{R}^n$ es una n -tupla ordenada de números reales:

$$\mathbf{v} = \begin{bmatrix} v_1 \\ v_2 \\ \vdots \\ v_n \end{bmatrix} \in \mathbb{R}^n \quad \text{o en forma transpuesta} \quad \mathbf{v}^T = [v_1, v_2, \dots, v_n] \quad (1)$$

2.1. Creación y Propiedades Básicas

En NumPy, un vector se crea generalmente con `np.array`. Es importante destacar que un arreglo unidimensional tiene forma `(n,)`, comportándose de manera fluida según la operación requerida:

```

1 # Definición de vectores u y v en  $\mathbb{R}^3$ 
2 u = np.array([2.0, 3.0, -1.0])
3 v = np.array([1.0, -1.0, 4.0])
4
```

```
5 print(f"Vector u: {u}, Dimension: {u.shape}, Tipo: {u.dtype}")
```

2.2. Suma Vectorial y Multiplicación por Escalar

Dadas $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ y un escalar $c \in \mathbb{R}$, las operaciones fundamentales son:

$$\mathbf{u} \pm \mathbf{v} = \begin{bmatrix} u_1 \pm v_1, & u_2 \pm v_2, & \dots, & u_n \pm v_n \end{bmatrix}^T \quad (2)$$

$$c\mathbf{u} = \begin{bmatrix} cu_1, & cu_2, & \dots, & cu_n \end{bmatrix}^T \quad (3)$$

```
1 suma = u + v          # [ 3.,  2.,  3.]
2 resta = u - v         # [ 1.,  4., -5.]
3 escalar = 3.0 * u     # [ 6.,  9., -3.]
```

2.3. Producto Escalar (Producto Punto)

El producto escalar euclidiano entre dos vectores $\mathbf{u}, \mathbf{v} \in \mathbb{R}^n$ genera un número real:

$$\mathbf{u} \cdot \mathbf{v} = \langle \mathbf{u}, \mathbf{v} \rangle = \sum_{i=1}^n u_i v_i = \mathbf{u}^T \mathbf{v} \quad (4)$$

En Python moderno (versión 3.5 en adelante), el operador infijo `@` es la forma estándar y preferida de calcular productos matriciales y vectoriales:

```
1 prod_punto_1 = np.dot(u, v)
2 prod_punto_2 = u @ v          # Operador @ estandar moderno
3 print("u . v =", prod_punto_2) # Resultado: -5.0
```

2.4. Norma Euclidiana y Magnitud

La norma L_2 (longitud o magnitud euclidiana) mide el tamaño del vector y se define como:

$$\|\mathbf{u}\|_2 = \sqrt{\mathbf{u} \cdot \mathbf{u}} = \sqrt{\sum_{i=1}^n u_i^2} \quad (5)$$

Generalizando, la norma L_p es $\|\mathbf{u}\|_p = (\sum |u_i|^p)^{1/p}$.

```
1 norma_u = np.linalg.norm(u)          # Norma L2: sqrt(2^2 + 3^2 + (-1)^2) = sqrt(14) ~ 3.7417
2 norma_L1 = np.linalg.norm(u, ord=1)  # Norma L1: |2| + |3| + |-1| = 6.0
3 norma_inf = np.linalg.norm(u, ord=np.inf) # Norma L_inf: max(|2|, |3|, |-1|) = 3.0
4
5 # Vector Unitario (Normalizacion): u_hat = u / ||u||
6 u_unitario = u / np.linalg.norm(u)
```

2.5. Ángulo entre Vectores y Proyección Ortogonal

El coseno del ángulo θ entre dos vectores no nulos viene dado por:

$$\cos \theta = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2} \implies \theta = \arccos \left(\frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{u}\|_2 \|\mathbf{v}\|_2} \right) \quad (6)$$

Dos vectores son ortogonales si y solo si $\mathbf{u} \cdot \mathbf{v} = 0$.

La proyección ortogonal de $\mathbf{u}$ sobre $\mathbf{v}$ se calcula como:

$$\text{proj}_{\mathbf{v}}(\mathbf{u}) = \frac{\mathbf{u} \cdot \mathbf{v}}{\|\mathbf{v}\|_2^2} \mathbf{v} \quad (7)$$

```

1 cos_theta = (u @ v) / (np.linalg.norm(u) * np.linalg.norm(v))
2 theta_rad = np.arccos(np.clip(cos_theta, -1.0, 1.0))
3 theta_deg = np.degrees(theta_rad)
4
5 # Proyeccion ortogonal de u sobre v
6 proj_v_u = ((u @ v) / (np.linalg.norm(v)**2)) * v
7 print(f"Angulo: {theta_deg:.2f} grados | Proyeccion: {proj_v_u}")

```

2.6. Producto Cruz (Vectorial en $\mathbb{R}^3$)

Para vectores en el espacio tridimensional $\mathbb{R}^3$, el producto vectorial produce un vector perpendicular a ambos:

$$\mathbf{u} \times \mathbf{v} = \begin{vmatrix} \mathbf{i} & \mathbf{j} & \mathbf{k} \\ u_1 & u_2 & u_3 \\ v_1 & v_2 & v_3 \end{vmatrix} = \begin{bmatrix} u_2 v_3 - u_3 v_2 \\ u_3 v_1 - u_1 v_3 \\ u_1 v_2 - u_2 v_1 \end{bmatrix} \quad (8)$$

```

1 cross_uv = np.cross(u, v)
2 print("u x v =", cross_uv) # [11., -9., -5.]
3 # Verificacion de ortogonalidad:
4 print("u . (u x v) =", u @ cross_uv) # 0.0 (exactamente ortogonal)

```

3. Matrices y Álgebra Matricial

Definición Matemática: Matriz

Una matriz $A \in \mathbb{R}^{m \times n}$ es un arreglo bidimensional de escalares organizado en m filas y n columnas:

$$A = \begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix} \quad (9)$$

3.1. Creación y Matrices Especiales

NumPy facilita la generación de estructuras matriciales clásicas:

```

1 A = np.array([[1.0, 2.0],
2               [3.0, 4.0]])
3
4 B = np.array([[5.0, 6.0],
5               [7.0, 8.0]])
6
7 # Matrices especiales:
8 I = np.eye(3) # Matriz identidad 3x3
9 O = np.zeros((2, 3)) # Matriz nula 2x3
10 U = np.ones((2, 2)) # Matriz de unos 2x2
11 D = np.diag([2, 5, -1]) # Matriz diagonal a partir de una lista

```

3.2. Multiplicación Matricial vs. Producto de Hadamard

⚠️Precaución Crítica de Sintaxis en Python!

En NumPy existen dos tipos completamente diferentes de productos:

1. **Producto Matricial** (AB): Se calcula mediante el operador `@` o `np.matmul(A, B)`. Requiere que el número de columnas de A coincida con el número de filas de B ($(m \times k) \times (k \times n) \rightarrow m \times n$).
2. **Producto de Hadamard** ($A \odot B$): Se calcula con el operador asterisco `A * B`. Corresponde a la multiplicación elemento por elemento ($c_{ij} = a_{ij}b_{ij}$) y exige matrices con formas idénticas.

La fórmula matemática formal del producto matricial es:

$$C_{ij} = (AB)_{ij} = \sum_{k=1}^p a_{ik}b_{kj} \quad (10)$$

```

1 # 1. Producto elemento a elemento (Hadamard):
2 hadamard = A * B
3 # [[1*5, 2*6], [3*7, 4*8]] = [[5, 12], [21, 32]]
4
5 # 2. Producto Matricial algebraico estandar:
6 prod_mat = A @ B
7 # [[1*5 + 2*7, 1*6 + 2*8], [3*5 + 4*7, 3*6 + 4*8]] = [[19, 22], [43,
8   50]]
9 print("Producto Matricial (A @ B):\n", prod_mat)
```

3.3. Matriz Transpuesta y Traza

La transpuesta A^T intercambia las filas por columnas: $(A^T)_{ij} = A_{ji}$. Una matriz es *simétrica* si $A = A^T$.

La traza $\text{tr}(A)$ es la suma de los elementos de la diagonal principal de una matriz cuadrada:

$$\text{tr}(A) = \sum_{i=1}^n a_{ii} \quad (11)$$

```

1 transpuesta = A.T           # 0 np.transpose(A)
2 traza = np.trace(A)         # 1.0 + 4.0 = 5.0
```

4. Determinantes, Rango e Invertibilidad

4.1. Determinante de una Matriz

Para una matriz cuadrada $A \in \mathbb{R}^{n \times n}$, el determinante $\det(A)$ o $|A|$ es un escalar que cuantifica la alteración de volumen provocada por la transformación lineal representada por A . Para una matriz 2×2 :

$$\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc \quad (12)$$

```

1 det_A = np.linalg.det(A)
2 print(f"det(A) = {det_A:.4f}") # 1*4 - 2*3 = -2.0
```

4.2. Rango de una Matriz e Independencia Lineal

El rango $\text{rg}(A)$ es el número máximo de columnas (o filas) linealmente independientes. En NumPy se utiliza la descomposición en valores singulares con un umbral de tolerancia numérica:

```
1 rango = np.linalg.matrix_rank(A)
2 print("Rango de A:", rango) # 2 (rango completo)
```

4.3. Inversa de una Matriz

Una matriz cuadrada A es no singular (invertible) si y solo si $\det(A) \neq 0$ o equivale a tener rango completo. La inversa A^{-1} satisface:

$$AA^{-1} = A^{-1}A = I \quad (13)$$

```
1 A_inv = np.linalg.inv(A)
2 print("Inversa de A:\n", A_inv)
3 # Comprobacion de que A @ A_inv es la identidad:
4 identidad_aprox = A @ A_inv
5 print("A @ A^(-1):\n", identidad_aprox)
```

```
Salida en Consola (Matriz Inversa):
Inversa de A:
[[-2.   1.]
 [ 1.5 -0.5]]
A @ A^(-1):
[[ 1.   0.]
 [ 0.   1.]]
```

5. Sistemas de Ecuaciones Lineales: $Ax = b$

Un sistema de m ecuaciones lineales con n incógnitas se escribe matricialmente como:

$$\begin{bmatrix} a_{11} & a_{12} & \dots & a_{1n} \\ a_{21} & a_{22} & \dots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \dots & a_{mn} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_m \end{bmatrix} \iff Ax = b \quad (14)$$

5.1. Resolución Directa y Eficiencia Algorítmica

Regla de Oro: Evite `np.linalg.inv(A) @ b`

Aunque matemáticamente $\mathbf{x} = A^{-1}\mathbf{b}$, en computación científica **nunca** se debe calcular la inversa explícitamente para resolver un sistema lineal. El cálculo explícito de A^{-1} es numéricamente inestable, más costoso en tiempo de cómputo ($O(n^3)$ con constante alta) y propenso a acumulación de errores de redondeo.

En su lugar, se debe utilizar siempre `np.linalg.solve(A, b)`, que internamente emplea factorización LU con pivoteo parcial (o Cholesky si la matriz es simétrica y definida positiva).

Consideremos el siguiente sistema 3×3 :

$$\begin{aligned} 2x_1 + x_2 - x_3 &= 8 \\ -3x_1 - x_2 + 2x_3 &= -11 \\ -2x_1 + x_2 + 2x_3 &= -3 \end{aligned}$$


```

1 A_sys = np.array([[ 2.0,  1.0, -1.0],
2                  [-3.0, -1.0,  2.0],
3                  [-2.0,  1.0,  2.0]])
4
5 b_sys = np.array([8.0, -11.0, -3.0])
6
7 # Resolución optima:
8 x = np.linalg.solve(A_sys, b_sys)
9 print("Vector solucion x:", x) # [ 2.,  3., -1.]
10
11 # Comprobacion con tolerancia numerica:
12 es_correcto = np.allclose(A_sys @ x, b_sys)
13 print("Es valida la solucion?:", es_correcto)

```

5.2. Condicionamiento Numérico

El número de condición $\kappa(A) = \|A\| \cdot \|A^{-1}\|$ indica cuán sensible es la solución del sistema a pequeñas perturbaciones en los datos:

```

1 kappa = np.linalg.cond(A_sys)
2 print(f"Numero de condicion: {kappa:.2f}")
3 # Si kappa es muy grande (ej. > 1e12), la matriz es mal condicionada o
   casi singular

```

5.3. Sistemas Sobredeterminados y Mínimos Cuadrados

Cuando tenemos más ecuaciones que incógnitas ($m > n$), generalmente no existe una solución exacta. Se busca el vector $\mathbf{x}^*$ que minimiza la norma del residuo: $\min_{\mathbf{x}} \|\mathbf{Ax} - \mathbf{b}\|_2^2$, cuya solución teórica cumple las ecuaciones normales $A^T \mathbf{Ax} = A^T \mathbf{b}$.

```

1 # Minimos cuadrados ordinarios (Ordinary Least Squares - OLS)
2 x_ls, residuos, rank, s = np.linalg.lstsq(A_sys, b_sys, rcond=None)

```

6. Valores y Vectores Propios (Autovalores y Autovectores)

Definición de Autovalores y Autovectores

Sea $A \in \mathbb{R}^{n \times n}$ una matriz cuadrada. Un escalar $\lambda \in \mathbb{C}$ es un **valor propio** (*eigenvalue*) y un vector no nulo $\mathbf{v} \in \mathbb{C}^n$ es su correspondiente **vector propio** (*eigenvector*) si satisfacen:

$$A\mathbf{v} = \lambda\mathbf{v} \iff (A - \lambda I)\mathbf{v} = \mathbf{0} \quad (15)$$

Para que existan soluciones no triviales ($\mathbf{v} \neq \mathbf{0}$), se debe cumplir la **ecuación característica**:

$$\det(A - \lambda I) = 0 \quad (16)$$

6.1. Cálculo en Python con `np.linalg.eig`

Sea la matriz:

$$A = \begin{bmatrix} 4 & -2 \\ 1 & 1 \end{bmatrix} \implies \det \begin{bmatrix} 4 - \lambda & -2 \\ 1 & 1 - \lambda \end{bmatrix} = \lambda^2 - 5\lambda + 6 = (\lambda - 3)(\lambda - 2) = 0 \quad (17)$$

Por tanto, los autovalores son $\lambda_1 = 3$ y $\lambda_2 = 2$.

```

1 A_eig = np.array([[4.0, -2.0],
2                  [1.0,  1.0]])
3
4 # Calculo de autovalores y autovectores:
5 autovalores, autovectores = np.linalg.eig(A_eig)
6
7 print("Autovalores:", autovalores)
8 # Nota: Cada columna autovectores[:, i] corresponde al autovalor
   autovalores[i]
9 print("Autovectores (en columnas):\n", autovectores)
10
11 # Verificacion A * v_0 = lambda_0 * v_0
12 v0 = autovectores[:, 0]
13 l0 = autovalores[0]
14 print("Comprobacion A @ v0 == l0 * v0:", np.allclose(A_eig @ v0, l0 * v0
   ))

```

```

Salida en Consola (Autovalores y Autovectores):
Autovalores: [3.  2.]
Autovectores (en columnas):
[[0.8944 0.7071]
 [0.4472 0.7071]]
Comprobacion A @ v0 == l0 * v0: True

```

6.2. Diagonalización de Matrices

Si una matriz A de $n \times n$ tiene n vectores propios linealmente independientes, se puede diagonalizar como:

$$A = V\Lambda V^{-1} \quad (18)$$

donde V es la matriz cuyas columnas son los autovectores y Λ es la matriz diagonal con los autovalores en su diagonal principal.

7. Descomposiciones Matriciales Fundamentales

Las factorizaciones o descomposiciones simplifican cálculos costosos transformando matrices arbitrarias en productos de matrices con propiedades favorables (triangulares, ortogonales o diagonales).

7.1. Descomposición LU

Factoriza una matriz cuadrada A en el producto de una matriz de permutación P , una triangular inferior unitaria L y una triangular superior U :

$$PA = LU \implies A = P^T LU \quad (19)$$

```

1 # Disponible en scipy.linalg
2 P, L, U = la.lu(A_sys)
3 print("Matriz L (Triangular Inferior):\n", L)
4 print("Matriz U (Triangular Superior):\n", U)

```

7.2. Descomposición QR

Factoriza cualquier matriz $A \in \mathbb{R}^{m \times n}$ en el producto de una matriz ortogonal Q ($Q^T Q = I$) y una matriz triangular superior R :

$$A = QR \quad (20)$$

Esta descomposición es fundamental en el cálculo de mínimos cuadrados y algoritmos iterativos para autovalores.

```
1 Q, R = np.linalg.qr(A_sys)
2 print("Q es ortogonal?", np.allclose(Q.T @ Q, np.eye(3)))
3 print("Reconstruccion Q @ R == A:", np.allclose(Q @ R, A_sys))
```

7.3. Descomposición en Valores Singulares (SVD)

Es considerada una de las herramientas más potentes del álgebra lineal moderna. Para cualquier matriz $A \in \mathbb{R}^{m \times n}$:

$$A = U \Sigma V^T \quad (21)$$

donde $U \in \mathbb{R}^{m \times m}$ y $V \in \mathbb{R}^{n \times n}$ son ortogonales, y $\Sigma \in \mathbb{R}^{m \times n}$ es una matriz diagonal con los valores singulares $\sigma_1 \geq \sigma_2 \geq \dots \geq 0$.

```
1 U_svd, S, Vt = np.linalg.svd(A_sys)
2 print("Valores singulares Sigma:", S)
3 # Reconstruccion: A = U @ diag(S) @ Vt
4 A_reconstruida = U_svd @ np.diag(S) @ Vt
5 print("Reconstruccion SVD exacta?", np.allclose(A_sys, A_reconstruida))
```

8. Tabla Resumen de Sintaxis y Operaciones

Operación Matemática	Sintaxis en Python	Descripción / Retorno
$\mathbf{u} + \mathbf{v}, \mathbf{u} - \mathbf{v}$	<code>u + v, u - v</code>	Suma y resta vectorial
$c\mathbf{v}$	<code>c * v</code>	Multiplicación de vector por escalar
$\mathbf{u} \cdot \mathbf{v}$	<code>u @ v</code> o <code>np.dot(u, v)</code>	Producto escalar
$\ \mathbf{v}\ _2$	<code>np.linalg.norm(v)</code>	Norma euclidiana (magnitud)
$\mathbf{u} \times \mathbf{v}$	<code>np.cross(u, v)</code>	Producto vectorial ($\mathbb{R}^3$)
$A \pm B$	<code>A + B, A - B</code>	Suma y diferencia matricial
$A \odot B$	<code>A * B</code>	Producto de Hadamard (elemento a elemento)
AB	<code>A @ B</code> o <code>np.matmul(A, B)</code>	Producto matricial estándar
A^T	<code>A.T</code> o <code>np.transpose(A)</code>	Matriz transpuesta
$\text{tr}(A)$	<code>np.trace(A)</code>	Traza de la matriz
$\det(A)$	<code>np.linalg.det(A)</code>	Determinante
$\text{rg}(A)$	<code>np.linalg.matrix_rank(A)</code>	Rango matricial
A^{-1}	<code>np.linalg.inv(A)</code>	Matriz inversa
$\kappa(A)$	<code>np.linalg.cond(A)</code>	Número de condición
$A\mathbf{x} = \mathbf{b}$	<code>np.linalg.solve(A, b)</code>	Solución directa de sistema lineal
$A\mathbf{v} = \lambda\mathbf{v}$	<code>np.linalg.eig(A)</code>	Autovalores y autovectores
$A = QR$	<code>np.linalg.qr(A)</code>	Descomposición QR
$A = U\Sigma V^T$	<code>np.linalg.svd(A)</code>	Descomposición en Valores Singulares (SVD)

Cuadro 1: Correspondencia entre el formalismo matemático y las primitivas computacionales de NumPy.

9. Buenas Prácticas Numéricas y Conclusiones

1. **Comparación con Tolerancia:** Los números de punto flotante (`float64`) están sujetos a errores de redondeo. Nunca compare resultados matriciales con igualdad estricta (`==`); use `np.allclose(A, B, rtol=1e-5, atol=1e-8)` o `np.isclose()`.
2. **Vectorización:** Aproveche las operaciones vectorizadas de NumPy; evite recorrer vectores o matrices mediante bucles iterativos manuales en Python.
3. **Uso del Operador @:** Favorezca el operador `@` para multiplicar matrices y vectores, reservando `*` exclusivamente para productos elemento a elemento.
4. **Ecosistema Ampliado:** Para cálculo simbólico exacto (fracciones racionales y variables analíticas), utilice la biblioteca **SymPy**; para matrices dispersas (*sparse matrices*) de gran escala, emplee `scipy.sparse`.